1. A banking system creates multiple account objects using different constructors (default, parameterized, and copy constructor).

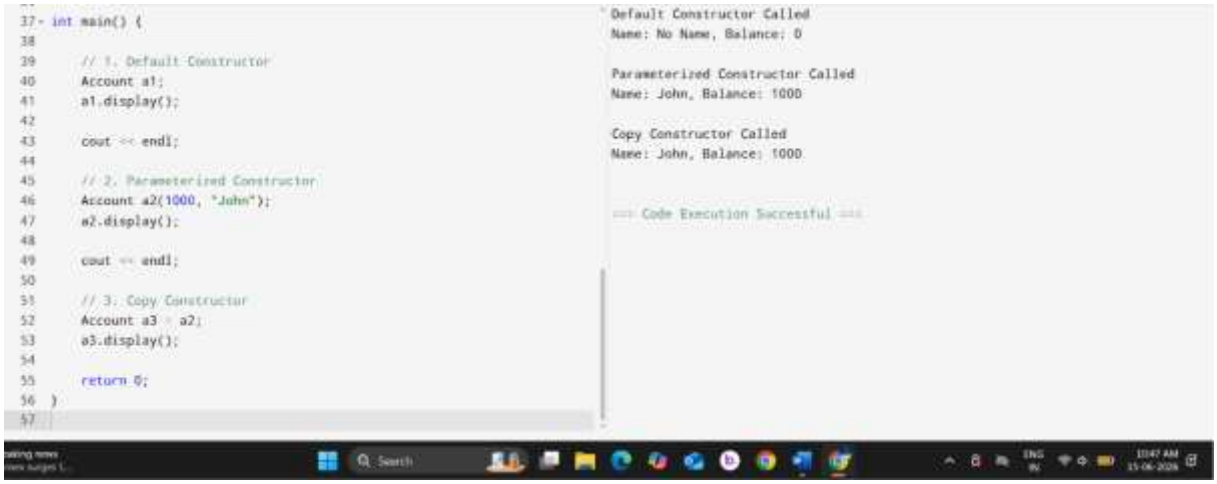
   - Analyze how object initialization differs in each case.
   - Predict the order in which constructors are called when an account object is created from another object.

PROGRAM

```
#include <iostream>
using namespace std;

class Account {
private:
   int balance;
   string name;

public:
   // Default Constructor
   Account() {
      balance = 0;
      name = "No Name";
      cout << "Default Constructor Called\n";
   }

   // Parameterized Constructor
   Account(int b, string n) {
      balance = b;
      name = n;
      cout << "Parameterized Constructor Called\n";
   }

   // Copy Constructor
   Account(const Account &a) {
      balance = a.balance;
      name = a.name;
      cout << "Copy Constructor Called\n";
   }

   // Display Function
   void display() {
      cout << "Name: " << name << ", Balance: " << balance << endl;
   }
};

int main() {

   // 1. Default Constructor
   Account a1;
   a1.display();
```

```
    cout << endl;

    // 2. Parameterized Constructor
    Account a2(1000, "John");
    a2.display();

    cout << endl;

    // 3. Copy Constructor
    Account a3 = a2;
    a3.display();

    return 0;
}
```

OUTPUT:



EXPLANATION:

Object initialization differs based on the type of constructor used. A default constructor initializes objects with predefined or default values when no arguments are provided. A parameterized constructor initializes objects using values passed by the user at the time of creation. A copy constructor initializes a new object by copying the values of an existing object, ensuring that both objects have the same data at initialization.

When an object is created from another object, first the original object is initialized using a constructor (usually parameterized or default). Then, the copy constructor is invoked to create the new object by copying the data from the existing object. Thus, the order is: constructor of original object followed by the copy constructor of the new object.

2. A software system manages inventory using a Product class that include A software system manages inventory using a Product class that includes:

    - Parameterized constructor for product details
    - Copy constructor for duplication
    - Destructor for releasing resources

- Overloaded + operator to combine stock quantities

Analyze:

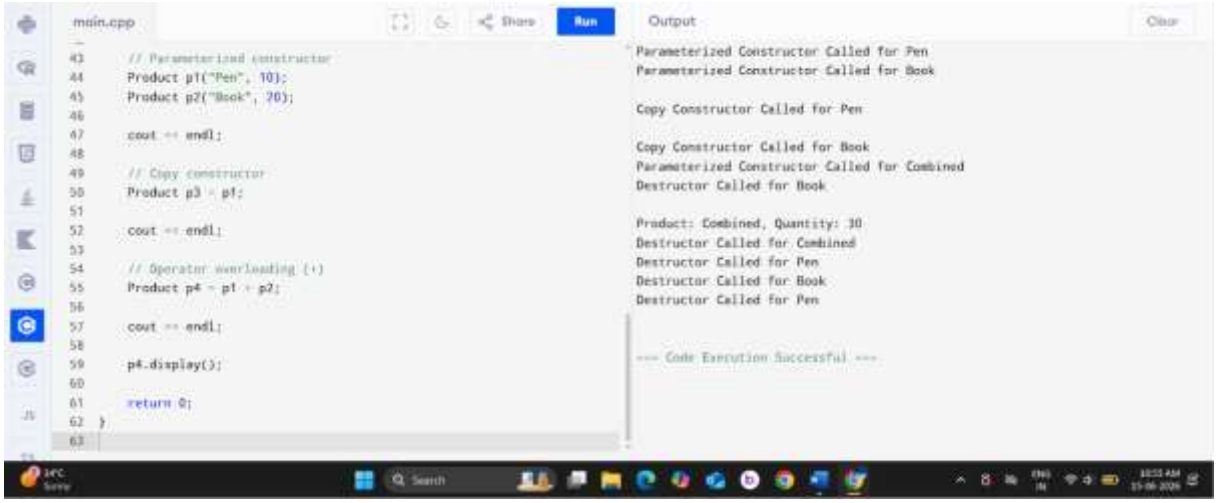
- The sequence of constructor/destructor calls when products are copied and combined.
- The role of operator overloading in simplifying inventory updates.
- Potential memory or logic issues if destructor is omitted.

PROGRAM

```
#include <iostream>
using namespace std;

class Product {
private:
   string name;
   int quantity;

public:
   // Parameterized Constructor
   Product(string n, int q) {
      name = n;
      quantity = q;
      cout << "Parameterized Constructor Called for " << name << endl;
   }

   // Copy Constructor
   Product(const Product &p) {
      name = p.name;
      quantity = p.quantity;
      cout << "Copy Constructor Called for " << name << endl;
   }

   // Destructor
   ~Product() {
      cout << "Destructor Called for " << name << endl;
   }

   // Operator Overloading (+)
   Product operator+(Product p2) {
      Product temp("Combined", 0);
      temp.quantity = this->quantity + p2.quantity;
      return temp;
   }

   void display() {
      cout << "Product: " << name << ", Quantity: " << quantity << endl;
   }
};

int main() {
```

```
    // Parameterized constructor
    Product p1("Pen", 10);
    Product p2("Book", 20);

    cout << endl;

    // Copy constructor
    Product p3 = p1;

    cout << endl;

    // Operator overloading (+)
    Product p4 = p1 + p2;

    cout << endl;

    p4.display();

    return 0;
}
```

OUTPUT:



EXPLANATION

When a Product object is created using a parameterized constructor, it initializes the product details. If another object is created from it, the copy constructor is invoked to duplicate the object's data. When two objects are combined using the overloaded + operator, a temporary object is created inside the operator function, which calls the constructor. After the operation, destructors are automatically called when objects go out of scope, and they execute in reverse order of object creation, including temporary objects.

Operator overloading simplifies inventory updates by allowing objects to be combined using natural expressions like p3 = p1 + p2. Instead of calling a separate function to add quantities, the + operator directly combines the stock of two products. This improves code readability,

makes it more intuitive, and closely resembles real-world operations, thereby enhancing program clarity and maintainability.

If the destructor is omitted, any dynamically allocated memory or resources such as file handles or connections may not be properly released when objects are destroyed. This can lead to memory leaks, inefficient resource usage, and potential program crashes over time. The destructor ensures that all resources are freed when an object goes out of scope, maintaining system stability and performance.

3. A ShoppingCart system uses:

- Parameterized constructor for product initialization
- Copy constructor for cart duplication
- Destructor for clearing cart memory
- Operator + to merge carts

Analyze object lifecycle and memory management during cart merging operations.

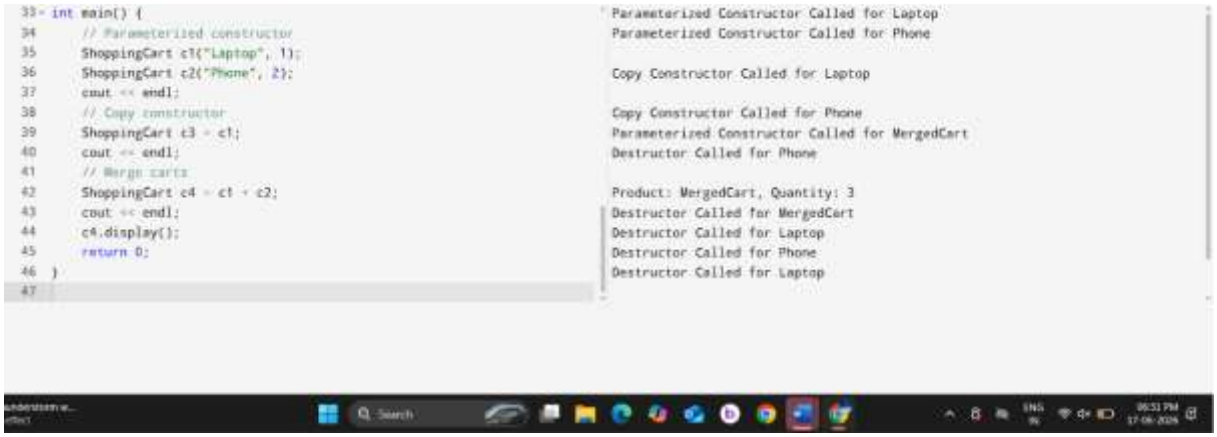
PROGRAM

```
#include <iostream>

using namespace std;

class ShoppingCart {

private:

   string productName;

   int quantity;

public:

   // Parameterized Constructor

   ShoppingCart(string name, int qty) {

      productName = name;

      quantity = qty;

      cout << "Parameterized Constructor Called for " << productName << endl;

   }

   // Copy Constructor

   ShoppingCart(const ShoppingCart &c) {
```

```
        productName = c.productName;
        quantity = c.quantity;
        cout << "Copy Constructor Called for " << productName << endl;
    }
    // Destructor
    ~ShoppingCart() {
        cout << "Destructor Called for " << productName << endl;
    }
    // Operator Overloading (+)
    ShoppingCart operator+(ShoppingCart c2) {
        ShoppingCart temp("MergedCart", 0);
        temp.quantity = this->quantity + c2.quantity;
        return temp;
    }
    void display() {
        cout << "Product: " << productName
             << ", Quantity: " << quantity << endl;   }};
int main() {
    // Parameterized constructor
    ShoppingCart c1("Laptop", 1);
    ShoppingCart c2("Phone", 2);
    cout << endl;
    // Copy constructor
    ShoppingCart c3 = c1;
    cout << endl;
```

```
    // Merge carts
    ShoppingCart c4 = c1 + c2;
    cout << endl;
    c4.display();
    return 0;
}
```

OUTPUT



EXPLANTION

The ShoppingCart system uses a parameterized constructor to initialize product details and a copy constructor to duplicate cart objects. The overloaded + operator allows easy merging of carts by combining product quantities using simple expressions. A destructor is used to release resources when objects go out of scope, ensuring proper memory management. Overall, the system demonstrates efficient object handling, code readability, and real-world functionality using OOP concepts in C++.

4. A retail company designs a class Product with the following features:

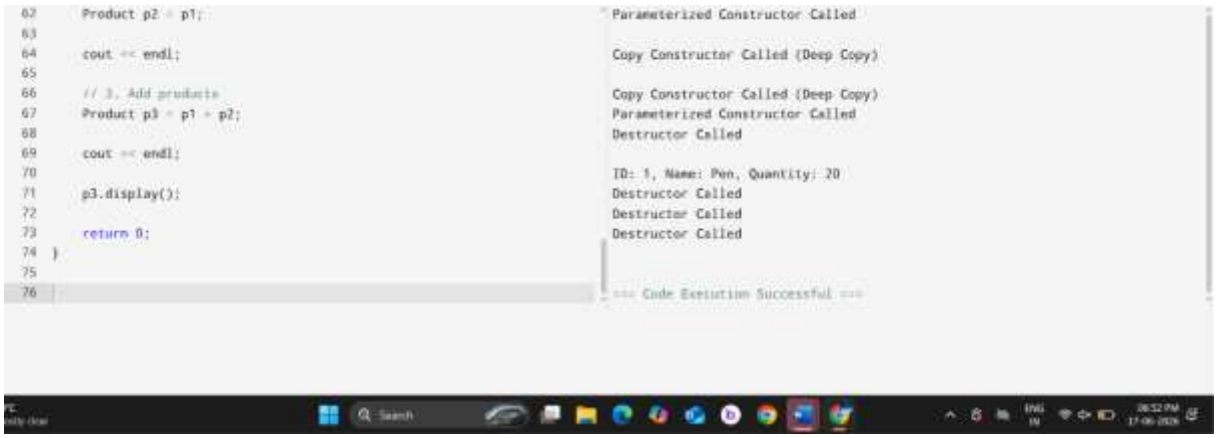
   Parameterized constructor to initialize product ID, name, and quantity
   Copy constructor to duplicate products during stock transfer
   Dynamic constructor to allocate memory for product name
   Destructor to release allocated memory
   Overloaded + operator to combine quantities of two products
   Tasks:
   Analyze the sequence of constructor and destructor calls when:
   A product is created, It is copied to another object, two products are added Explain how deep copy ensures correct behavior during stock duplication.

PROGRAM

```
#include <iostream>
#include <cstring>
using namespace std;

class Product {
private:
   int id;
   char *name;     // dynamic memory
   int quantity;

public:
   // Parameterized Constructor (Dynamic)
   Product(int i, const char *n, int q) {
      id = i;
      quantity = q;

      name = new char[strlen(n) + 1];  // allocate memory
      strcpy(name, n);

      cout << "Parameterized Constructor Called\n";
   }

   // Copy Constructor (Deep Copy)
   Product(const Product &p) {
      id = p.id;
      quantity = p.quantity;

      name = new char[strlen(p.name) + 1];  // new memory
      strcpy(name, p.name);

      cout << "Copy Constructor Called (Deep Copy)\n";
   }

   // Operator Overloading (+)
   Product operator+(Product p2) {
      Product temp(id, name, quantity + p2.quantity);
      return temp;
   }

   // Display
   void display() {
      cout << "ID: " << id
           << ", Name: " << name
           << ", Quantity: " << quantity << endl;
   }

   // Destructor
   ~Product() {
```

```
        delete[] name;   // free memory
        cout << "Destructor Called\n";
    }
};

int main() {

    // 1. Create product
    Product p1(1, "Pen", 10);

    cout << endl;

    // 2. Copy product
    Product p2 = p1;

    cout << endl;

    // 3. Add products
    Product p3 = p1 + p2;

    cout << endl;

    p3.display();

    return 0;
}
```

OUTPUT



EXPLANATION

When a product is created using the parameterized constructor, it initializes the product ID, name (with dynamic memory allocation), and quantity. When this object is copied to another object, the copy constructor is invoked, which performs a deep copy of all data members. When two products are added using the overloaded + operator, a temporary object is created inside the operator function, calling the constructor again. After program execution, destructors are automatically called in reverse order of object creation, including temporary objects, ensuring that dynamically allocated memory is properly released.

Deep copy ensures that each object has its own separate memory for dynamically allocated members like the product name. Instead of copying just the pointer (which would lead to shared memory), the copy constructor allocates new memory and copies the actual data. This prevents issues such as unintended modification of one object affecting another and avoids double deletion errors during destruction. As a result, stock duplication works correctly and safely without memory conflicts.

5. Consider banking application defines a class Account with:
Multiple constructor overloads (default, account number only, full details)
A copy constructor for transaction logs
A destructor for closing accounts
Overloaded operators:
+ → deposit
- → withdrawal
Tasks:

1. Analyze how constructor overloading resolves initialization ambiguity.
2. Trace object creation when:
    - Account is passed by value to a function
    - Account is returned from a function
3. Evaluate risks if:
    - Copy constructor is not defined
    - Destructor is not invoked properly
4. Analyze how operator overloading simplifies financial transactions logic.

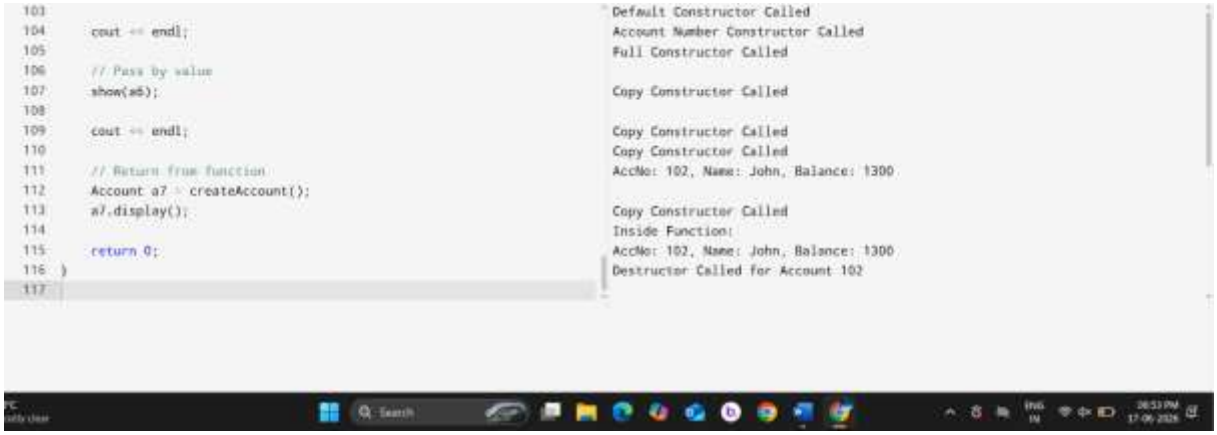
PROGRAM

```
#include <iostream>
using namespace std;

class Account {
private:
   int accNo;
   string name;
   double balance;

public:
   // 1. Default Constructor
   Account() {
      accNo = 0;
      name = "No Name";
      balance = 0;
      cout << "Default Constructor Called\n";
   }

   // 2. Constructor with Account Number only
   Account(int acc) {
      accNo = acc;
      name = "Unknown";
      balance = 0;
      cout << "Account Number Constructor Called\n";
```

```
    }

    // 3. Full Parameterized Constructor
    Account(int acc, string n, double bal) {
        accNo = acc;
        name = n;
        balance = bal;
        cout << "Full Constructor Called\n";
    }

    // 4. Copy Constructor
    Account(const Account &a) {
        accNo = a.accNo;
        name = a.name;
        balance = a.balance;
        cout << "Copy Constructor Called\n";
    }

    // 5. Destructor
    ~Account() {
        cout << "Destructor Called for Account " << accNo << endl;
    }

    // 6. Deposit (+)
    Account operator+(double amount) {
        Account temp = *this;
        temp.balance += amount;
        return temp;
    }

    // 7. Withdrawal (-)
    Account operator-(double amount) {
        Account temp = *this;
        if (amount <= temp.balance)
            temp.balance -= amount;
        else
            cout << "Insufficient Balance\n";
        return temp;
    }

    void display() {
        cout << "AccNo: " << accNo
             << ", Name: " << name
             << ", Balance: " << balance << endl;
    }
};

// Function (pass by value)
void show(Account a) {
    cout << "Inside Function:\n";
```

```
    a.display();
}

// Function returning object
Account createAccount() {
    Account temp(200, "TempUser", 5000);
    return temp;
}

int main() {

    // Constructor overloading
    Account a1;
    Account a2(101);
    Account a3(102, "John", 1000);

    cout << endl;

    // Copy constructor
    Account a4 = a3;

    cout << endl;

    // Operator overloading
    Account a5 = a3 + 500;   // deposit
    Account a6 = a5 - 200;   // withdrawal

    a6.display();

    cout << endl;

    // Pass by value
    show(a6);

    cout << endl;

    // Return from function
    Account a7 = createAccount();
    a7.display();

    return 0;
}
```

OUTPUT



EXPLANATION

1. Constructor overloading resolves initialization ambiguity by providing multiple constructors with different parameter lists. Depending on the arguments passed during object creation, the compiler automatically selects the appropriate constructor. This allows objects to be initialized in different ways, such as with default values, partial details, or complete information, ensuring flexibility and eliminating confusion during object initialization.

2.

(a) When passed by value to a function

- When an Account object is passed by value, a copy of the object is created, which invokes the copy constructor. This ensures that the function operates on a separate copy without affecting the original object.

(b) When returned from a function

- When an Account object is returned from a function, a temporary object is created and then copied to the receiving object using the copy constructor. However, modern compilers may optimize this process using copy elision, reducing unnecessary copies.

3.

(a) If copy constructor is not defined

- If a custom copy constructor is not defined, the compiler generates a default shallow copy. This can lead to issues such as shared memory, unintended modifications, and double deletion errors if dynamic memory is involved.

(b) If destructor is not invoked properly

- If the destructor is not executed properly, resources such as memory or files may not be released, leading to memory leaks, reduced performance, and potential system crashes.

4. Operator overloading simplifies financial transactions by allowing intuitive expressions like acc = acc + 1000 for deposit and acc = acc - 500 for withdrawal. This makes the code more readable and natural compared to traditional function calls, improving clarity and making the program resemble real-world banking operations.